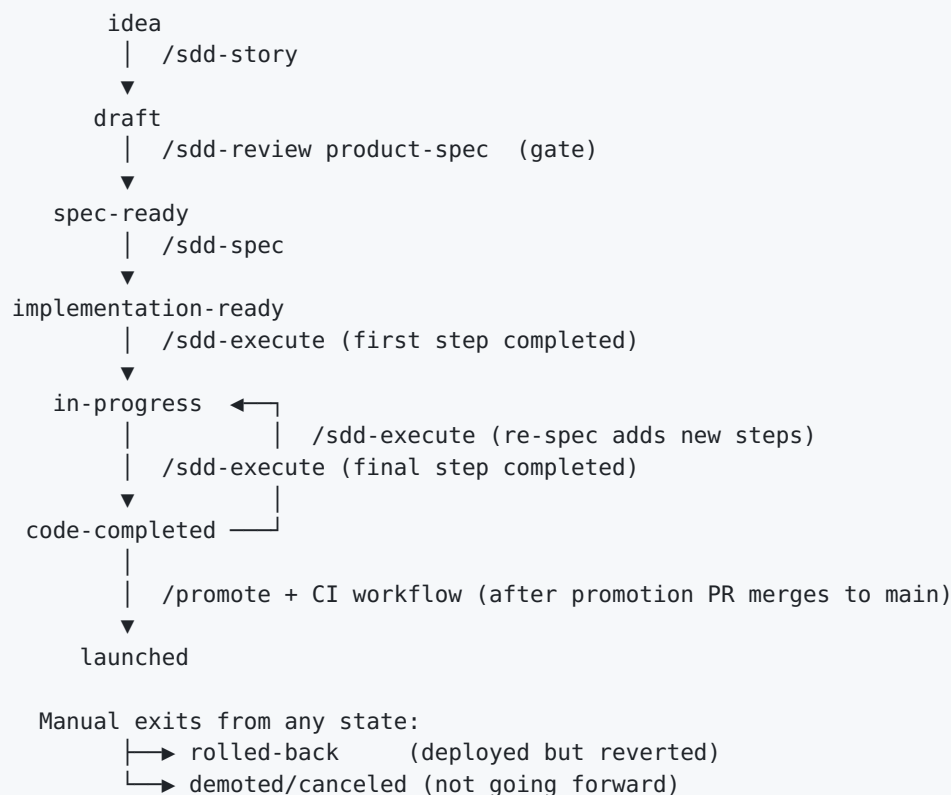


SDD Feature Lifecycle — Status Transitions

Companion reference to `sdd-flow.pdf`. Where the flow document covers the five phases at a narrative level, this document is the **state machine**: every lifecycle status, every legal transition, the skill or CI workflow that performs the flip, and what gets written to `feature.md` at each step.

This is the structure that lets a new agent session — or a new human contributor — reconstruct exactly where a feature stands without reading any conversation history.

Quick Map



The Nine Statuses

Every feature lives in exactly one of these states. The status is the single source of truth — CI workflows, the `/sdd-status` skill, and the GitHub PR description all read it from `feature.md`.

Status	Meaning	What exists on disk	Who writes it
idea	Story captured, no spec yet. Rare — most features go straight to draft .	feature.md (cover sheet only)	Manual or /sdd-story
draft	Product spec written; awaiting AI review.	feature.md + product-spec.md	/sdd-story
spec-ready	Product spec approved by /sdd-review . Ready for implementation planning.	Same as draft	/sdd-review product-spec (gate)
implementation-ready	Implementation spec generated with numbered steps and grep-cited file paths.	feature.md + product-spec.md + implementation-spec.md	/sdd-spec
in-progress	Execution started — at least one step done, at least one step pending.	All four files (feature.md , product-spec.md , implementation-spec.md , context.md)	/sdd-execute (first step completion)
code-completed	All steps done. Awaiting final integration PR and promotion.	All four files	/sdd-execute (last step completion)
launched	Live in production. Promotion PR merged to main .	All four files + **Committed to main** SHA + **Launched date**	CI: ci-validate-feature-status.yml
rolled-back	Was deployed, then reverted.	All four files + revert note in context.md	Manual
demoted/canceled	Not going forward. May or may not have any spec written.	Whatever exists	Manual

Bug-Specific Fields

Bugs use the same nine statuses but feature.md carries extra headers:

Field	Values	Set by
Type	bug (features omit this field or set it to feature)	/sdd-triage
Severity	SEV-1 , SEV-2 , SEV-3	/sdd-triage
GitHub Issue	URL of the originating issue	/sdd-triage

/sdd-triage routes bugs into one of three tracks: - **Track A — Hotfix.** SEV-1. Branches from main , PRs to main , back-merged to main-dev . Bypasses the SDD lifecycle entirely; appended to docs/runbooks/hotfix-log.md instead. - **Track B — Config-only.** SEV-2 or SEV-3 that's fixable by changing a config value. Follows docs/runbooks/config-rollout.md . No feature.md created. - **Track C — SDD path.** Anything else. Creates a feature directory and follows the standard nine-state lifecycle below, with ****Type****: bug set on feature.md .

The state machine that follows applies to features and Track-C bugs identically.

Transition 1: idea → draft

Field	Value
Trigger skill	/sdd-story <slug> [story text]
Side effects	Creates docs/roadmap/features/NNN-<slug>/feature.md and product-spec.md . NNN is auto-assigned by counting existing dirs.
Status History row appended	\ YYYY-MM-DD \ idea → draft \ /sdd-story \ Product spec generated \
Files written	feature.md (cover sheet, lifecycle = draft , branch = feature/<slug>), product-spec.md (requirements with FR-1, FR-2, ..., governance gates, acceptance criteria)
Files read	docs/runbooks/reviewer-registry.md (to populate the Reviewers snapshot section), docs/runbooks/feature-workflow.md (governance fields)
Branch operations	None — /sdd-story is doc-only. The feature branch is created later in /sdd-execute .

Note: the cover sheet on feature.md always lists ****Lifecycle Status****: draft after this transition. There is no actual idea row written to disk — idea is a conceptual placeholder for "a user has the idea but hasn't run /sdd-story yet."

Transition 2: `draft` → `spec-ready`

Field	Value
Trigger skill	<code>/sdd-review <slug> product-spec</code>
Type	Gate — this is the first of two AI review gates in the lifecycle. The transition is blocked until the review passes.
Side effects	Updates <code>feature.md</code> <code>lifecycle</code> field. Appends a status history row. May write review notes into <code>context.md</code> .
Files read	<code>product-spec.md</code> (full), <code>feature.md</code> , optionally other active features for overlap detection
Decision criteria	The reviewer agent checks: (a) all FR items are testable, (b) acceptance criteria are concrete, (c) governance gates are correctly identified (proto changes? config keys? DB migrations?), (d) no overlap with another in-progress feature.
On approval	Lifecycle flips to <code>spec-ready</code> . Status history row appended: <code>\ YYYY-MM-DD \ draft → spec-ready \ /sdd-review \ Product spec approved (N advisory warnings) \ </code>
On rejection	Lifecycle stays <code>draft</code> . Reviewer comments appended to <code>product-spec.md</code> . User updates the product spec, then re-runs <code>/sdd-review</code> .
Re-run safety	If lifecycle is already <code>spec-ready</code> or later, the skill asks: "Product spec is already approved (status: <code><status></code>). Re-run review anyway? (yes / no)"

Why this is a hard gate. The next phase, `/sdd-spec`, runs as a `general-purpose` sub-agent with **high effort** — that's expensive in tokens and time. Gating it on a product-spec review catches half-baked requirements before they consume planner cycles.

Transition 3: spec-ready → implementation-ready

Field	Value
Trigger skill	<code>/sdd-spec <slug></code>
Side effects	Creates <code>implementation-spec.md</code> with numbered steps. Each step cites concrete file paths and symbol names found via <code>grep</code> . Updates <code>feature.md</code> Reviewers snapshot (per-step Reviewers attached based on step categories).
Status History row appended	<code>\ YYYY-MM-DD \ spec-ready → implementation-ready \ /sdd-spec \ Implementation spec generated with N steps \ </code>
Files read	<code>product-spec.md</code> , <code>docs/runbooks/reviewer-registry.md</code> , the entire affected-services subtree
Files written	<code>implementation-spec.md</code> — numbered steps with <code>**Status**</code> : <code>pending</code> , file paths, symbol names, line ranges, verification commands
Hard rule (prompt-enforced)	"Every step you write must cite evidence found in the codebase via <code>Read</code> , <code>find</code> , or <code>grep</code> . Never invent a file path, function name, struct name, or line number."
Optional gate	<code>/sdd-review <slug> impl-spec</code> — advisory only (does not flip lifecycle). Useful for overlap checks against other in-progress features.

Re-spec mid-flight. It is legal to re-run `/sdd-spec` after execution has started. The skill preserves the `done` status on already-completed steps and only adds new steps for any product-spec changes since the last spec. Common scenario: product-spec gets an FR added at session N+2 after step 5 of 7 is already merged. Re-running `/sdd-spec` adds steps 8–10 without re-doing 1–5. This is recorded in the status history as e.g. `\| 2026-05-11 \| in-progress (re-spec) \| /sdd-spec \| Implementation spec regenerated with 11 steps (preserved Step 1 done; added Steps 10–11 for FR-9/FR-10) \|` .

Transition 4: `implementation-ready` → `in-progress`

Field	Value
Trigger skill	<code>/sdd-execute <slug> [step-number\ next\ all]</code> — first successful step completion
Side effects	Step 1 is completed (or whichever step the user invoked). <code>implementation-spec.md</code> step status flips from <code>pending</code> to <code>done</code> . Per-step PR opened against <code>feature/<slug></code> . Append-only entry written to <code>context.md</code> .
Status History row appended	<code>\ YYYY-MM-DD \ implementation-ready → in-progress \ /sdd-execute \ Step N complete (<short description>) \ </code>
Files read at session start (boot sequence)	<code>feature.md</code> (status check), <code>implementation-spec.md</code> (current state), <code>context.md</code> (every prior session's decisions)
Branch operations	Creates <code>feature/<slug></code> from <code>origin/main-dev</code> if not already present. Creates <code>feature/<slug>/step-N</code> for the step. Pushes step branch. Opens PR <code>feature/<slug>/step-N → feature/<slug></code> .
Confirmation gate	Before any write, the executor prints the planned changes and stops with "Type 'go' to proceed." No step writes silently.

The four mandatory writes per step: 1. **Code edits.** What the spec said to do. 2. **`implementation-spec.md`** — step Status flipped from `pending` to `done`. 3. **`context.md`** — append-only entry: what was done, decisions made, files modified, next step. 4. **`feature.md`** — status flipped (only on first and last step), status history row appended (only on first and last step).

Transition 5: `in-progress` → `code-completed`

Field	Value
Trigger skill	<code>/sdd-execute <slug></code> — when the last pending step completes
Side effects	Final step PR opened. <code>implementation-spec.md</code> has zero pending steps. <code>context.md</code> entry notes "all steps complete; open integration PR."
Status History row appended	<code>\ YYYY-MM-DD \ in-progress → code-completed \ /sdd-execute \ Step N complete (final). Integration PR ready. \ </code>
Files read	Same as transition 4 — boot sequence reads <code>feature.md</code> , <code>implementation-spec.md</code> , <code>context.md</code> .
What's left to do	Open the final integration PR: <code>feature/<slug> → main-dev</code> . This is not automatic — the user runs <code>gh pr create --base main-dev --head feature/<slug></code> (or the equivalent in the GitHub web UI). The status history records: <code>\ YYYY-MM-DD \ code-completed → Final PR \ /sdd-execute \ Integration PR #NNN created: feature/<slug> → main-dev \ </code>

Why `code-completed` is its own status. Two reasons: 1. There can be a meaningful delay between "all step PRs merged into `feature/<slug>` " and "integration PR merged into `main-dev` ." During that window, the feature is code-complete but not yet on the dev trunk. 2. `/promote` reads `code-completed` specifically when building the promotion PR description — it lists every `code-completed` feature being promoted in the body, separately for features and bugs.

Transition 6: `code-completed` → `launched`

Field	Value
Trigger	CI workflow <code>.github/workflows/ci-validate-feature-status.yml</code> , triggered on any push to <code>main</code> .
Detection	Workflow checks the merge commit message for <code>release: promote</code> . If found, it parses the PR body for feature slugs at <code>code-completed</code> .
Side effects (automatic)	For each promoted feature: 1. Lifecycle flipped: <code>code-completed</code> → <code>launched</code> 2. <code>**Committed to main**</code> : <code><sha></code> field set on <code>feature.md</code> 3. <code>**Launched date**</code> : <code>YYYY-MM-DD</code> field set on <code>feature.md</code> 4. Status history row appended: <code>\ YYYY-MM-DD \ code-completed → launched \ /promote + CI \ Promoted via PR #NNN; committed SHA-HASH to main \ </code> 5. <code>context.md</code> session entry appended: <code>## Session YYYY-MM-DD (CI: feature status automation)</code>
Commit + push	CI commits the changes back to <code>main</code> with a <code>chore: auto-update feature statuses for promotion PR #NNN</code> message.
Manual fallback	If the CI workflow fails (rare): run <code>/promote</code> on the <code>main</code> branch — same logic, manual trigger.

This is the single source of truth for "is it live in production?" Searching `git grep launched docs/roadmap/features/*/feature.md` returns every launched feature. Searching for a specific commit SHA in `**Committed to main**` fields reveals which feature shipped in which release.

Transition 7 (manual): Any → rolled-back

Field	Value
Trigger	Manual — by the human operator after a production revert.
When it happens	A <code>launched</code> feature is reverted via a hotfix or a <code>git revert</code> PR that lands on <code>main</code> .
Side effects	Operator edits <code>feature.md</code> : 1. Lifecycle: <code>launched</code> → <code>rolled-back</code> 2. Status history row: <code>\ YYYY-MM-DD \ launched → rolled-back \ <operator></code> <code>\ Reverted by PR #NNN; reason: <one-line> \ </code> 3. <code>context.md</code> entry: <code>### Session YYYY-MM-DD (rollback)</code> with the full reason and any follow-up plan.
Subsequent paths	A rolled-back feature can be re-launched after a fix: bump lifecycle back to <code>code-completed</code> , redo Transition 6 on the next promotion.

Transition 8 (manual): Any → demoted/canceled

Field	Value
Trigger	Manual — by the human operator.
When it happens	The feature is decided against, deprioritized indefinitely, or superseded by another feature.
Side effects	Lifecycle flipped to <code>demoted/canceled</code> . Status history row records reason. <code>context.md</code> entry explains the decision. Branch may be deleted (optional).
What's kept	All artifacts stay on disk as historical record. <code>/sdd-review</code> later treats <code>demoted/canceled</code> features specially when scanning for duplicates against new feature ideas.

Re-entry Loops

Loop A: `in-progress` ↔ `in-progress` (re-spec)

If a product spec changes (FRs added, scope reduced) after execution has started:

1. User updates `product-spec.md` directly (or re-runs `/sdd-story` to overwrite).

- 2. User runs `/sdd-spec` again. The planner preserves any step already marked `done` and adds new steps for the changed/added FRs.
- 3. Lifecycle stays at `in-progress`. Status history row records: `\| YYYY-MM-DD \| in-progress (re-spec) \| /sdd-spec \| Implementation spec regenerated with N steps (preserved Steps 1–M done; added Steps M+1–N) \|`.
- 4. `/sdd-execute` resumes from the first new `pending` step.

Loop B: `in-progress` step retry

If a step fails verification (test fails, lint fails, manual rejection):

- 1. `/sdd-execute` does **not** open a PR. The step status stays `pending` (or moves to `blocked`).
- 2. Lifecycle stays `in-progress`.
- 3. `context.md` records the failure cause.
- 4. User re-runs `/sdd-execute <slug> N` to retry the same step.

Loop C: `spec-ready` ↔ `draft`

If `/sdd-review product-spec` rejects, lifecycle stays `draft`. User edits `product-spec.md` and re-runs the review. No artificial intermediate state.

Status History Table — The Audit Trail

Every transition writes one row to a table at the top of `feature.md`:

```
## Status History

| Date | Status | Updated by | Note |
| --- | --- | --- | --- |
| 2026-05-10 | `idea` → `draft` | /sdd-story | Product spec generated |
| 2026-05-10 | `draft` → `spec-ready` | /sdd-review | Product spec approved (1 advisory warning) |
| 2026-05-10 | `spec-ready` → `implementation-ready` | /sdd-spec | Implementation spec generated with 9 steps |
| 2026-05-11 | `implementation-ready` → `in-progress` | /sdd-execute | Step 1 complete (docker-compose.yml) |
| 2026-05-11 | `in-progress` (re-spec) | /sdd-spec | Implementation spec regenerated with 11 steps |
| 2026-05-11 | `in-progress` (unchanged) | /sdd-execute | Step 7 complete (secret-scan CI job + .gitleaks.yml) |
| 2026-05-11 | `in-progress` → `code-completed` | /sdd-execute | Step 10 complete; Step 11 skipped |
| 2026-05-11 | `code-completed` → Final PR | /sdd-execute | Integration PR #157 created |
| 2026-05-11 | `code-completed` → `launched` | /promote + CI | Promoted via PR #158; committed 89e07ef to main
```

The above is the actual table from feature `004-make-repo-public-secure` — the one that hardened this repo for the public launch. Every row corresponds to one transition above. The table is the per-feature audit trail; combined with `context.md`'s session log, it answers any "what happened, when, and why" question about a feature.

Lifecycle Field Discipline

Three rules keep the field honest in the long run:

1. **feature.md is the only place the lifecycle field is written.** The status history table at the top of the file is the canonical record. No external trackers, no Jira, no GitHub project boards mirror this field.
 2. **/sdd-status is read-only.** It reads `feature.md` files via `git show origin/feature/<slug>:feature.md` (falling back to `origin/main-dev` if no feature branch exists). It never writes anything. This means anyone — human or agent — can run `/sdd-status` to get an authoritative live view without side effects.
 3. **CI flips the final transition, not a human.** The most error-prone manual transition — `code-completed` → `launched` — is automated. If a human forgets to update the file after a promotion, the next `git push origin main` triggers `ci-validate-feature-status.yml` to catch up.
-

Reading a Feature Directory Cold

Drop into any feature directory and you can reconstruct the full state without any conversation history:

File	What it tells you
<code>feature.md</code>	The current lifecycle status, the audit trail, who reviews it, what branch it lives on.
<code>product-spec.md</code>	What was asked for. The functional requirements. The governance gates that apply.
<code>implementation-spec.md</code>	How it's being built. Numbered steps with file/line evidence. Per-step status (<code>done</code> , <code>pending</code> , <code>blocked</code> , <code>skipped</code>).
<code>context.md</code>	Every session log entry, append-only. Decisions made, deviations from the spec, files modified, what the next session should do.

The lifecycle status on `feature.md` is the index. Everything else flows from there.

What Generalizes

The pattern is portable. To apply this lifecycle to another spine-pattern monorepo:

1. **Copy** `.claude/skills/sdd-*` into the target repo.
2. **Adapt the reviewer registry** (`docs/runbooks/reviewer-registry.md`) to your team and services.
3. **Adapt the governance gates** in `/sdd-story` 's product-spec template to the breaking-change classes your platform cares about (proto, schema, config, API).

4. **Adopt the same nine-state lifecycle** — `idea` through `launched` with two manual exits. The transitions and audit trail format work as-is.

5. **Wire `ci-validate-feature-status.yml`** to your promotion convention. The pattern detects merge commits whose messages match a regex; adjust the regex.

The result: the same property holds — every feature has a directory, every directory has an auditable lifecycle, every promotion updates statuses automatically, and any new agent or contributor can reconstruct exactly where things stand from the checked-in artifacts alone.